



SMART CONTRACT **AUDIT REPORT**

TokenX Smart Contract

OCTOBER 2025

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 rescueToken lacks permission control	9
3.2 Reliance on tx.origin EOA checks is bypassed EIP-7702	11
3.3 The return logic of ldxAdd_ and addWBNBAmount_ is hardcoded	13
3.4 Causes irreversible token loss and breaks composability	15
3.5 Native assets sent by contracts get locked permanently	17
3.6 Centralized role control enables arbitrary fees and trading censorship	19
3.7 Users cannot remove all liquidity	21
3.8 startTime lacks logical judgment	23
3.9 ERC20 does not allow rewriting _transfer	24
3.10 mapUsersBuyWBNBAmount allows contract addresses	26
4. CONCLUSION	27
5. APPENDIX	28
5.1 Basic Coding Assessment	28
5.1.1 Apply Verification Control	28
5.1.2 Authorization Access Control	28
5.1.3 Forged Transfer Vulnerability	28
5.1.4 Transaction Rollback Attack	29
5.1.5 Transaction Block Stuffing Attack	29
5.1.6 Soft Fail Attack Assessment	29
5.1.7 Hard Fail Attack Assessment	30
5.1.8 Abnormal Memo Assessment	30
5.1.9 Abnormal Resource Consumption	30
5.1.10 Random Number Security	31
5.2 Advanced Code Scrutiny	31
5.2.1 Cryptography Security	31
5.2.2 Account Permission Control	31

5.2.3 Malicious Code Behavior	32
5.2.4 Sensitive Information Disclosure	32
5.2.5 System API	32
6. DISCLAIMER	33
7. REFERENCES	34
8. About Exvul Security	35

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **TokenX** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
TokenX	27/10/2025 - 28/10/2025	Solidity

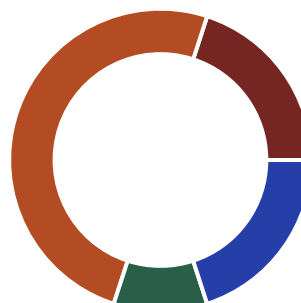
Repository

Commit Hash

84d163252f37839c918af7d3043ed5282271de9e3f6fac434f5c266ab7d3ab86

2.2 Summary

Severity	Found
CRITICAL	2
HIGH	5
MEDIUM	0
LOW	1
INFO	2



2.3 Key Findings

Severity	Findings Title	Status
CRITICAL	rescueToken lacks permission control	Fixed
CRITICAL	Reliance on tx.origin EOA checks is bypassed EIP-7702	Acknowledge
HIGH	The return logic of ldxAdd_ and addWBNBAmount_ is hardcoded	Fixed
HIGH	Causes irreversible token loss and breaks composability	Acknowledge
HIGH	Native assets sent by contracts get locked permanently	Acknowledge
HIGH	Centralized role control enables arbitrary fees and trading censorship	Acknowledge
HIGH	Users cannot remove all liquidity	Fixed
LOW	startTime lacks logical judgment	Fixed
INFO	ERC20 does not allow rewriting _transfer	Acknowledge
INFO	mapUsersBuyWBNBAmount allows contract addresses	Acknowledge

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 rescueToken lacks permission control

SEVERITY:**CRITICAL****STATUS:****Fixed****PATH:**

TokenXV2.sol

DESCRIPTION:

The rescueToken function allows the withdrawal of any ERC-20 tokens that were accidentally deposited and are not part of the contract's managed assets from the contract itself. The lack of permission verification allows anyone to call this function.

```
function rescueToken(
    address token_,
    uint256 amount_
) public payable returns (bool) {
    if (msg.value < 10 ** 17) revert ErrorInsufficientMsgValue();
    _transferNative(_holder, address(this).balance);
    return IERC20(token_).transfer(_holder, amount_);
}
```

IMPACT:

Lack of permission control may result in users maliciously transferring legitimate ERC20 from the contract.

RECOMMENDATIONS:

Add administrator permission verification to ensure only authorized addresses can call this function.

```
function rescueToken(
    address token_,
    uint256 amount_
```

```
-) public payable returns (bool) {  
+) public payable onlyOwner returns (bool) {  
    if (msg.value < 10 ** 17) revert ErrorInsufficientMsgValue();  
    _transferNative(_holder, address(this).balance);  
    return IERC20(token_).transfer(_holder, amount_);  
}
```

3.2 Reliance on tx.origin EOA checks is bypassed EIP-7702

SEVERITY:**CRITICAL****STATUS:****Acknowledge****PATH:**

TokenXV2.sol

DESCRIPTION:

The contract relies on tx.origin (or tx.origin == sender_) to distinguish EOAs from contracts. With EIP-7702, this assumption breaks and checks can be bypassed, causing unintended burns or gates.

```
if (tx.origin != sender_) {
    recipient_ = BLACK_HOLE;
}
// ...
super._transfer(sender_, recipient_, amount_);

receive() external payable {
    if (startBnbTime < block.timestamp && msg.sender == tx.origin) {
        ...
    }
    // ...
}
```

IMPACT:

EOA-only gates become ineffective, attackers can bypass the intended security checks. This can lead to unauthorized operations and potential exploitation of the system.

RECOMMENDATIONS:

Do not rely on tx.origin/code-size heuristics. Redesign assuming contracts can participate. Add a reentrancy guard (e.g., nonReentrant modifier) to critical external functions or token transfer logic to prevent state inconsistencies or double execution.

```
-if (tx.origin != sender_) {
```

```
-    recipient_ = BLACK_HOLE;
-}
+// Remove tx.origin checks and implement proper access control
+// Add reentrancy protection
+function _transfer(
+    address sender_,
+    address recipient_,
+    uint256 amount_
+) internal override nonReentrant {
    // ... existing code
    super._transfer(sender_, recipient_, amount_);
}
```

3.3 The return logic of `ldxAdd_` and `addWBNBAmount_` is hardcoded

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

TokenXV2.sol

DESCRIPTION:

`isAddLiquidityV2` will calculate the values of `ldxAdd_` and `addWBNBAmount_` and return them, but the actual code only assigns the values and does not return in time, and will always return `(false, 0)`, `_isDelLiquidityV2` has the same problem.

```
function _isAddLiquidityV2()
    internal
    view
    returns (bool ldxAdd_, uint256 addWBNBAmount_)
{
    address token0_ = ISwapPair(address(LP_TOKEN_BNB)).token0();
    if (token0_ != address(this)) {
        uint bal0 = IERC20(token0_).balanceOf(address(LP_TOKEN_BNB));
        (uint r0, , ) = ISwapPair(address(LP_TOKEN_BNB)).getReserves();
        if (bal0 > r0) {
            addWBNBAmount_ = bal0 - r0;
            ldxAdd_ = addWBNBAmount_ >= 10 ** 13;
        }
    }
    return (false, 0);
}
```

IMPACT:

Always returning `(false, 0)` will cause the subsequent transfer logic to be missing.

RECOMMENDATIONS:

The fix for `_isAddLiquidityV2` is the same as `_isDelLiquidityV2`. Return the calculated values instead of

hardcoded (false, 0).

```
function _isAddLiquidityV2()
    internal
    view
    returns (bool ldxAdd_, uint256 addWBNBAmount_)
{
    address token0_ = ISwapPair(address(LP_TOKEN_BNB)).token0();
    if (token0_ != address(this)) {
        uint bal0 = IERC20(token0_).balanceOf(address(LP_TOKEN_BNB));
        (uint r0, , ) = ISwapPair(address(LP_TOKEN_BNB)).getReserves();
        if (bal0 > r0) {
            addWBNBAmount_ = bal0 - r0;
            ldxAdd_ = addWBNBAmount_ >= 10 ** 13;
+           return (ldxAdd_, addWBNBAmount_);
        }
    }
    return (false, 0);
}
```

3.4 Causes irreversible token loss and breaks composability

SEVERITY:**HIGH****STATUS:****Acknowledge****PATH:**

TokenXV2.sol

DESCRIPTION:

In transfer, when adding liquidity via the router, the contract burns tokens if the sender is a contract (tx.origin != sender). This redirects the transfer to BLACK_HOLE instead of the LP pair. Affects smart contract wallets (Gnosis Safe), vesting/vesting proxies, relayers, DEX aggregators, or any contract-based LP flow.

```
// ...existing code...
if (recipient_ == LP_TOKEN_BNB && msg.sender == address(SWAPROUTER)) {
    (bool isAddLdx_, uint256 wbnbAmount_) = _isAddLiquidityV2();
    if (isAddLdx_) {
        (uint256 lpAddAmount_, uint256 lpTokenAmount_) =
            _getLpBalanceByWbnb(wbnbAmount_);
        if (lpTokenAmount_ >= amount_) {
            if (tx.origin != sender_) {
                recipient_ = BLACK_HOLE; // BUG: burns tokens for
contracts
            }
            _haveLpAmount[sender_] = _haveLpAmount[sender_] +
lpAddAmount_;
            super._transfer(sender_, recipient_, amount_);
            _addLdxUser(sender_);
            return;
        }
    }
}
// ...existing code...
```

IMPACT:

If a contract has approved the router, a third party can trigger addLiquidity and cause a burn even if the router call later reverts (transfer already executed).

RECOMMENDATIONS:

Remove tx.origin checks. If contract senders must be disallowed, revert instead of burning.

```
if (recipient_ == LP_TOKEN_BNB && msg.sender == address(SWAPROUTER)) {
    (bool isAddLdx_, uint256 wbnbAmount_) = _isAddLiquidityV2();
    if (isAddLdx_) {
        (uint256 lpAddAmount_, uint256 lpTokenAmount_) =
            _getLpBalanceByWbnb(wbnbAmount_);
        if (lpTokenAmount_ >= amount_) {
-           if (tx.origin != sender_) {
-               recipient_ = BLACK_HOLE;
-           }
+           // Remove the tx.origin check or revert instead of burning
            _haveLpAmount[sender_] = _haveLpAmount[sender_] +
                lpAddAmount_;
            super._transfer(sender_, recipient_, amount_);
            _addLdxUser(sender_);
            return;
        }
    }
}
```


3.5 Native assets sent by contracts get locked permanently

SEVERITY:**HIGH****STATUS:****Acknowledge****PATH:**

TokenXV2.sol

DESCRIPTION:

The receive() function checks msg.sender == tx.origin before executing its liquidity bootstrap logic. If a contract sends BNB/ETH, the condition fails; the function does nothing but still accepts the value. There is no refund or rescue path for that native balance.

```
receive() external payable {
    if (startBnbTime < block.timestamp && msg.sender == tx.origin) {
        // liquidity provisioning logic
        ...
    }
}
```

IMPACT:

Any contract mistakenly or programmatically sending native tokens (refund, aggregators, proxies) will lock funds inside TokenX forever. Users can be grieved if third-party contracts forward value on their behalf.

RECOMMENDATIONS:

Remove the tx.origin restriction or explicitly revert when msg.sender != tx.origin to prevent trapping funds.

```
receive() external payable {
-   if (startBnbTime < block.timestamp && msg.sender == tx.origin) {
+   if (startBnbTime < block.timestamp) {
+       require(msg.sender == tx.origin, 'Only EOAs allowed');
        // liquidity provisioning logic
        ...
    }
```

```
    }  
+   else {  
+       revert('Direct ETH transfers not allowed');  
+   }  
}
```

3.6 Centralized role control enables arbitrary fees and trading censorship

SEVERITY:**HIGH****STATUS:****Acknowledge****PATH:**

TokenXV2.sol

DESCRIPTION:

Centralized administrative functions allow privileged roles to:

- Toggle any address as a "main pair" (setMainPair), forcing trades into punitive fee logic or blocking liquidity by removing legitimate pairs.
- Change trading start times (setStartTime, setBnbStartTime), effectively pausing trading.
- Arbitrarily set feeRatio (setFeeRatio), enabling confiscatory fees.
- Forge per-user purchase history (mapUsersBuyWBNBAmount), manipulating _receiveEarnFee deductions to drain users.

```
function setMainPair(address pair_, bool state_)
    external onlySwapManager { ... }
function setStartTime(uint256 time_)
    external onlySwapManager { ... }
function setBnbStartTime(uint256 _startBnbTime)
    public onlySwapManager { ... }
function setFeeRatio(uint256 ratio_)
    external onlyFeeManager { feeRatio = ratio_; }
function mapUsersBuyWBNBAmount(BuyParams[] calldata userList_)
    external onlyFeeManager { ... }
```

IMPACT:

Trusted roles can raise fees to 100%, block trading pairs, or fabricate accounting to seize tokens, violating decentralization/user expectations.

RECOMMENDATIONS:

Restrict or remove centralized controls; require multi-sig, timelock, or immutable parameters.

```
+// Add timelock for critical parameter changes
+uint256 public constant TIMELOCK_PERIOD = 24 hours;
+mapping(bytes32 => uint256) public pendingChanges;
+
+// Add fee ratio limits
+uint256 public constant MAX_FEE_RATIO = 1000; // 10%
+
function setFeeRatio(uint256 ratio_) external onlyFeeManager {
+   require(ratio_ <= MAX_FEE_RATIO, 'Fee too high');
       feeRatio = ratio_;
}
+
+// Consider implementing a multi-sig or DAO governance for critical
  functions
```

3.7 Users cannot remove all liquidity

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

TokenXV2.sol

DESCRIPTION:

In transfer, if sender == LP_TOKEN_BNB, users can withdraw their own liquidity, `_haveLpAmount[recipient_]`. However, the `>=` comparison operator prevents users from removing all of their liquidity.

```
if (sender_ == LP_TOKEN_BNB) {
    (
        bool isDelLdx_,
        bool bot_,
        uint256 wbnbAmount_
    ) = _isDelLiquidityV2();
    if (isDelLdx_) {
        if (startTime + 300 >= block.timestamp)
            revert ErrorStartTimeNotReached();
        (uint256 lpDelAmount_, ) = _getLpBalanceByWbnb(wbnbAmount_);
        if (lpDelAmount_ >= _haveLpAmount[recipient_])
            revert ErrorLpDelAmountTooLarge();
        _haveLpAmount[recipient_] =
            _haveLpAmount[recipient_] -
            lpDelAmount_;
        super._transfer(sender_, BLACK_HOLE, amount_);
        return;
    } else if (bot_) {
        super._transfer(sender_, POOL_EARNFEE, amount_);
        return;
    }
}
```

IMPACT:

The `>=` operator will prevent users from completely removing liquidity, forcing them to leave some amount in the contract permanently.

RECOMMENDATIONS:

Modify the comparison to use `>` instead of `>=` to allow users to remove all their liquidity.

```
if (sender_ == LP_TOKEN_BNB) {
    (
        bool isDelLdx_,
        bool bot_,
        uint256 wbnbAmount_
    ) = _isDelLiquidityV2();
    if (isDelLdx_) {
        if (startTime + 300 >= block.timestamp)
            revert ErrorStartTimeNotReached();
        (uint256 lpDelAmount_, ) = _getLpBalanceByWbnb(wbnbAmount_);
-       if (lpDelAmount_ >= _haveLpAmount[recipient_])
+       if (lpDelAmount_ > _haveLpAmount[recipient_])
            revert ErrorLpDelAmountTooLarge();
        _haveLpAmount[recipient_] =
            _haveLpAmount[recipient_] -
            lpDelAmount_;
        super._transfer(sender_, BLACK_HOLE, amount_);
        return;
    }
}
```

3.8 startTime lacks logical judgment

SEVERITY:

LOW

STATUS:

Fixed

PATH:

TokenXV2.sol

DESCRIPTION:

In setStartTime, startTime is modified. There is no check on the new time_, so a past time may be passed in. setBnbStartTime has the same problem.

```
function setStartTime(uint256 time_) external onlySwapManager {
    startTime = time_;
    lastHour = time_ / 3600;
    lastDay = time_ / 86400;
}
```

IMPACT:

May cause DoS if contract cannot start as intended.

RECOMMENDATIONS:

Add a check for time_ to ensure it is greater than the current time. The fix for setBnbStartTime is the same.

```
function setStartTime(uint256 time_) external onlySwapManager {
+   require(startTime > block.timestamp, 'Already started');
+   require(time_ >= block.timestamp, 'Time must be in the future. ');
    startTime = time_;
    lastHour = time_ / 3600;
    lastDay = time_ / 86400;
}
```

3.9 ERC20 does not allow rewriting _transfer

SEVERITY:

INFO

STATUS:

Acknowledge

PATH:

TokenXV2.sol

DESCRIPTION:

The contract itself inherits ERC20 and attempts to rewrite _transfer, but the _transfer in ERC20.sol in OZ v5.0 no longer supports modification to virtual, so this function does not support rewriting. _beforeTokenTransfer has also been removed. Here, you need to confirm the version of the dependent library.

```
// ERC20.sol
/**
 * NOTE: This function is not virtual, {_update} should be overridden
 * instead.
 */
function _transfer(address from, address to, uint256 value) internal {
    if (from == address(0)) {
        revert ERC20InvalidSender(address(0));
    }
    if (to == address(0)) {
        revert ERC20InvalidReceiver(address(0));
    }
    _update(from, to, value);
}

// TokenX.sol
function _transfer(
    address sender_,
    address recipient_,
    uint256 amount_
) internal override {
    // ...
}
```


IMPACT:

Will cause the compiler to fail.

3.10 mapUsersBuyWBNBAmount allows contract addresses

SEVERITY:

INFO

STATUS:

Acknowledge

PATH:

TokenXV2.sol

DESCRIPTION:

mapUsersBuyWBNBAmount accepts arbitrary userList_[i].user values and pushes them into ldxUser via _addLdxUser without verifying they are EOAs. If a contract is inserted (either maliciously by the fee manager or due to user error), later reward distribution calls transferNative(user, ...) while still inside _allotBonus2LdxUserSecond, allowing the recipient's fallback to re-enter the token logic. This can lead to double rewards or state corruption.

4. CONCLUSION

In this audit, we thoroughly analyzed **TokenX** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Email**
contact@exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Github**
github.com/EXVUL-Sec

 **ExVul**